

CS121 Data Structures in C

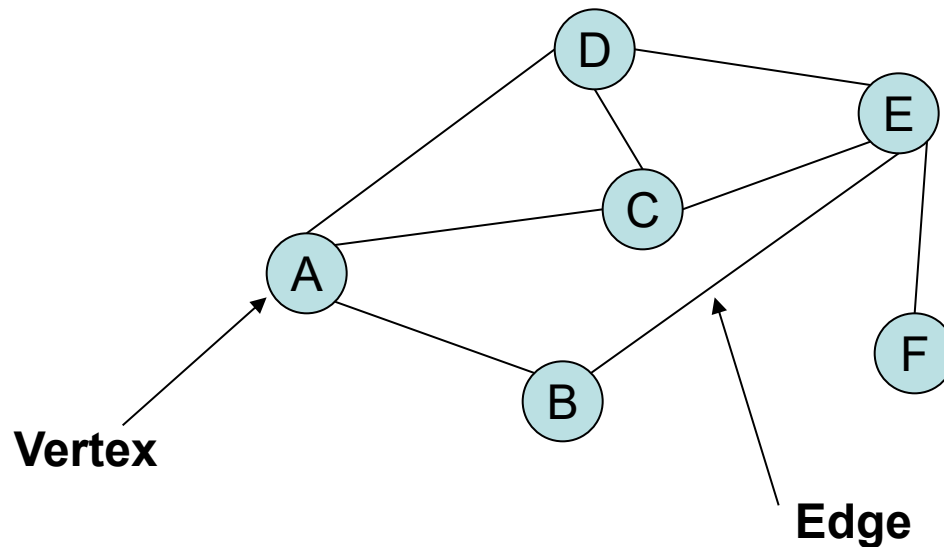
13 *Graph*

Outline

- Graph
 - ♦ Definition and terminology
- Graph representation
- Topological sort

Graphs

- Extremely useful tool in modeling problems.
- Consist of:
 - ♦ Vertices
 - ♦ Edges, sometimes referred as *arcs*



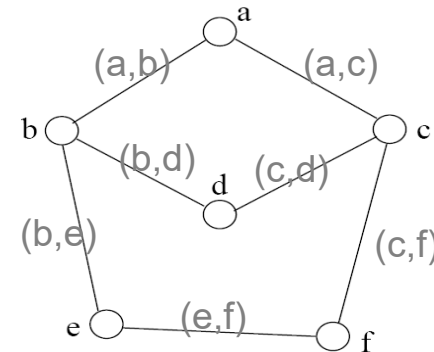
Vertices can be considered “sites” or locations.

Edges represent connections between vertices.

Definition

- A graph $G=(V, E)$ consists of a set of vertices, V , and a set of edges, E .
 - ♦ Each edge is a pair of vertices (v, w) , where v and w belong to V .
- Undirected graph
 - ♦ If the pair is unordered, the graph is **undirected**;
- Directed graph
 - ♦ If the pair is ordered, the graph is **directed** (*digraphs for short*).

An undirected graph



$$V=\{ a, b, c, d, e, f \}$$

$$E=\{ (a,b), (a,c), (b,d), (c,d), (b,e), (c,f), (e,f) \}$$

Terminology

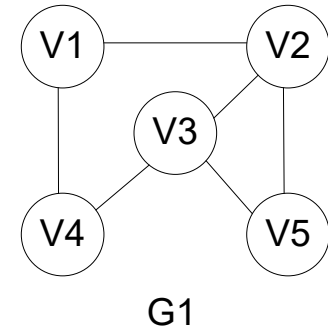
- Adjacent
 - ♦ If v_1 and v_2 are connected, they are said to be **adjacent** vertices.
 - v_1 and v_2 are endpoints of the edge (v_1, v_2) .
 - ♦ In an undirected graph, edge (w, v) , and hence (v, w)
 - w is adjacent to v and v is adjacent to w .
- Weight or cost
 - ♦ Third component of edge: *weight* or *cost*
- Path and length
 - ♦ A *path* in a graph is a sequence of vertices $w_1, w_2, \dots, w_N$
 - ♦ The *length* of such a path is the number of edges on the path, which is equal to $N-1$.

If we are talking about **directed graphs**, where edges have **direction**. This means that (v_1, v_2) does not equal to (v_2, v_1) . Directed graphs are drawn with arrows (called arcs) between edges.

 This means (A, B) only, not (B, A)

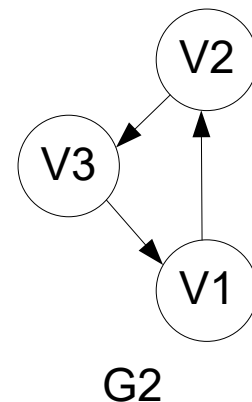
Terminology

- Connected
 - ♦ An undirected graph is *connected* if there is a path from every vertex to every other vertex.



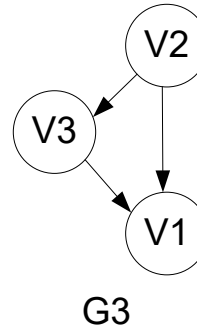
- Strongly connected
 - ♦ A directed graph with this property is called *strongly connected*.

For example: $V_1 \rightarrow V_2$, $V_2 \rightarrow V_1$



Terminology

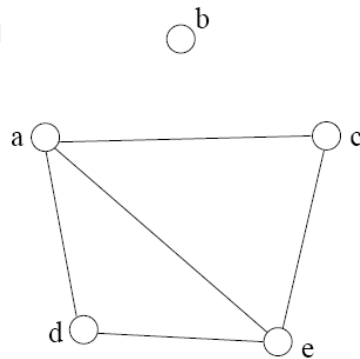
- Weakly connected
 - ♦ If a directed graph is not strongly connected, but the underlying graph (without direction to the arcs) is connected, the graph is said to be *weakly connected*.



- Complete graph
 - ♦ A *complete graph* is a graph in which there is an edge between every pair of vertices.
 - (1): complete undirected graph
 - (2): complete directed graph

Adjacency Matrix

undirected graph

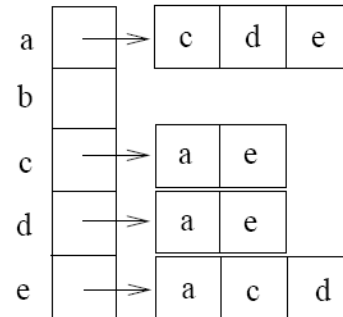
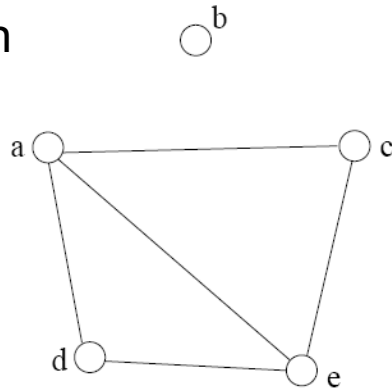


	a	b	c	d	e
a	0	0	1	1	1
b	0	0	0	0	0
c	1	0	0	0	1
d	1	0	0	0	1
e	1	0	1	1	0

- 2D array $A[0..n-1][0..n-1]$, where n is the number of vertices in a graph.
- Each row and column is indexed by the vertex id
 - ♦ e.g. $a=0, b=1, c=2, d=3, e=4$.
- If there is an undirected edge connecting vertices i and j :
 $A[i][j]=1$ and $A[j][i]=1$;
otherwise,
 $A[i][j]=A[j][i]=0$.

Adjacency List

undirected graph



For **sparse** graph

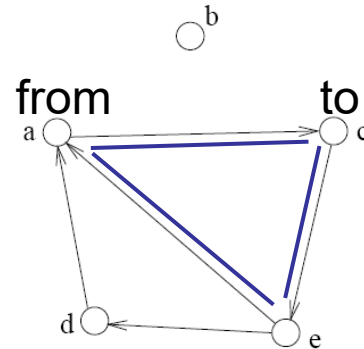
- The adjacency list is a 1D array $A[0..n-1]$ of linked lists, where n is the number of vertices in a graph.
- Each array entry is indexed by the vertex id.
- Each list $A[i]$ stores the ids of the vertices adjacent to vertex i .

Directed Graph

- The adjacency matrix and the adjacency list.

1. Adjacency Matrix

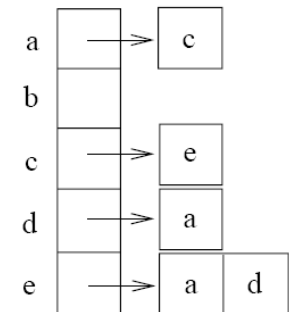
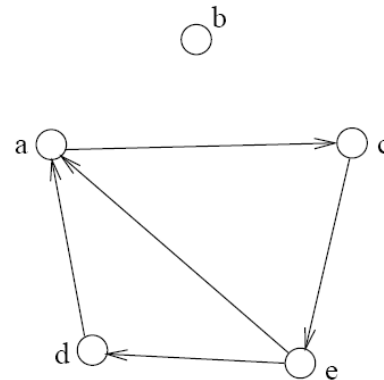
- 2D array $A[0..n-1][0..n-1]$, where n is the number of vertices in a graph.
- If there is an directed edge from vertices i to j : $A[i][j]=1$;
- otherwise, $A[i][j]=0$.



	a	b	c	d	e
a	0	0	1	0	0
b	0	0	0	0	0
c	0	0	0	0	1
d	1	0	0	0	0
e	1	0	0	1	0

2. Adjacency List

- The adjacency list is a 1D array $A[0..n-1]$ of linked lists, where n is the number of vertices in a graph.
- Each array entry is indexed by the vertex id.
- If there is an directed edge from vertices i to j , then list $A[i]$ stores j .



Adjacency List vs. Matrix

- **Adjacency List**

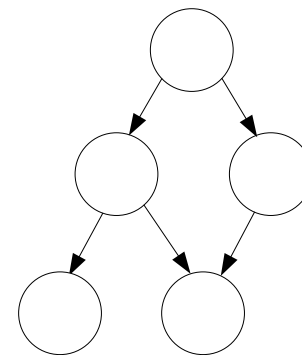
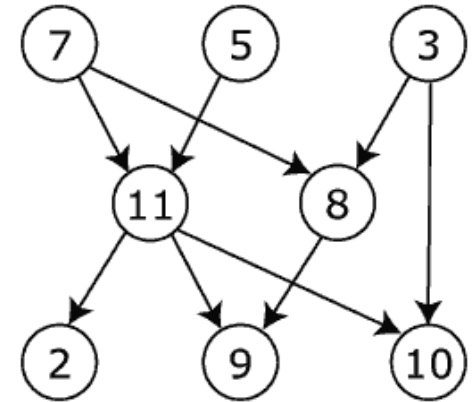
- ◆ More compact than adjacency matrix if a graph has few edges.
- ◆ Requires more time to find if an edge exists.

- **Adjacency Matrix**

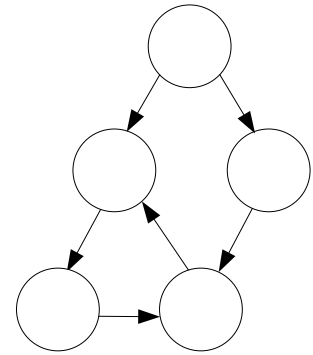
- ◆ Always require $\Theta(n^2)$ space.
 - This can waste a lot of space if the number of edges are small (the matrix is sparse).
- ◆ Can quickly find if an edge exists.

Directed Acyclic Graph

- A **directed path** is a sequence of vertices $(v_0, v_1, \dots, v_k)$.
 - ♦ Such that (v_i, v_{i+1}) is an *arc*.
- A **directed cycle** is a directed path such that the first and last vertices are the same.
- A directed acyclic graph is a directed graph with no directed cycles.



G1: DAG

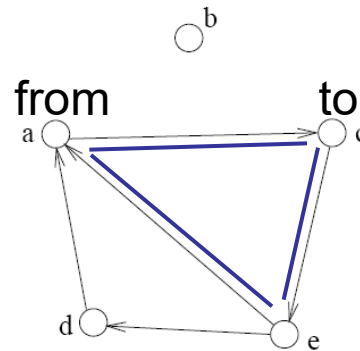


G2:
Directed
Graph

Representation

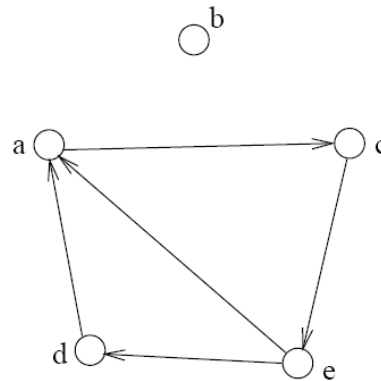
- The adjacency matrix and the adjacency list can be used.

1. Adjacency Matrix



	a	b	c	d	e
a	0	0	1	0	0
b	0	0	0	0	0
c	0	0	0	0	1
d	1	0	0	0	0
e	1	0	0	1	0

2. Adjacency List



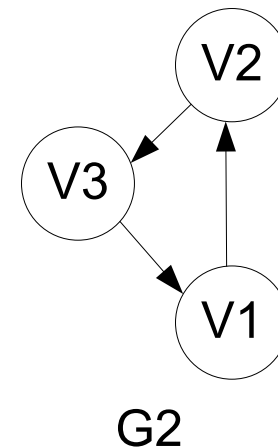
a	→	c
b		
c	→	e
d	→	a
e	→	a d

Indegree and Outdegree

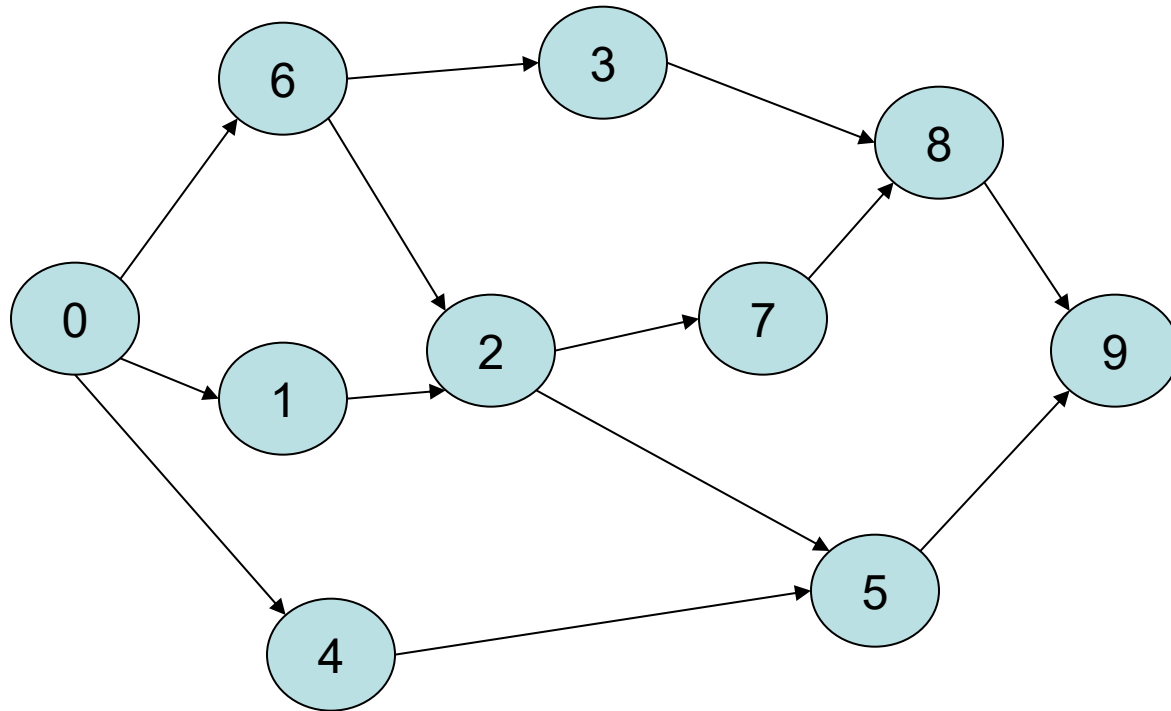
- Since the arcs are directed
 - ♦ We can't simply talk about $\deg(v)$.
- Instead, we need to consider the arcs **coming “in”** and **going “out”**.
 - ♦ Thus, we define terms **Indegree(v)**, and **Outdegree(v)**.
- Each **arc(u,v)** contributes 1 count to the outdegree of u and the indegree of v.

$$\sum_{\text{vertex } v} \text{indegree}(v) = \sum_{\text{vertex } v} \text{outdegree}(v) = m$$

m is the number of edges in a graph.



Example



Indeg(2)? 2

Indeg(8)? 2

Outdeg(0)? 3

Number of arcs?
 $m = 13$

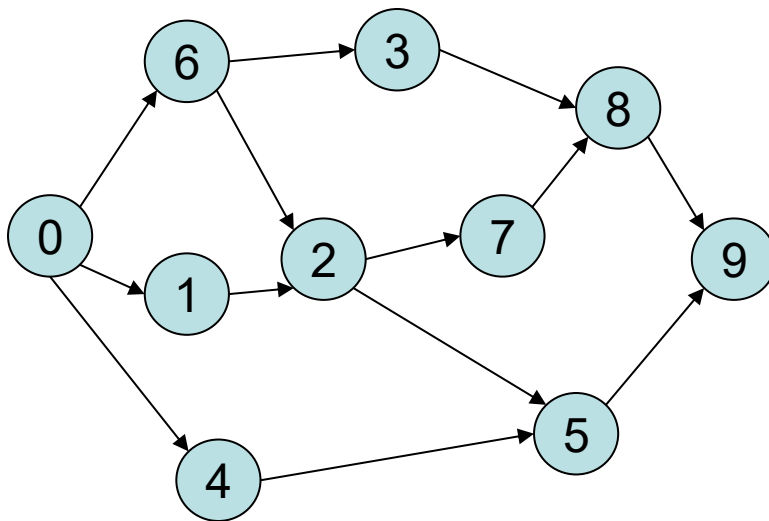
Total outdegree? 13

Total indegree? 13

Topological Sort

Topological Sort

- Topological sort is an algorithm for a directed acyclic graph.
 - ♦ An ordering of vertices in a directed acyclic graph, such that if there is a path from v_i to v_j , then v_j appears after v_i in the ordering.



For example:

0, 1, 2, 5, 9

0, 4, 5, 9

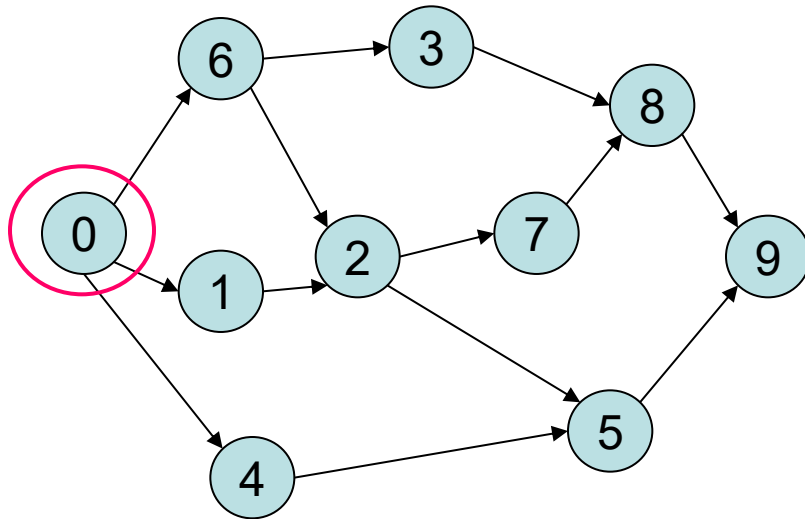
0, 6, 3, 7 ?

Topological Sort Algorithm

- Algorithm

1. Find any vertex with zero indegree, that is, no incoming edges.
2. If it is a vertex i , delete all i 's outgoing arcs.
The outgoing **arcs** (i, j) are no longer useful, since tasks j does not need to wait for i anymore- so remove all i 's outgoing arcs.
3. With vertex i removed, the new graph is still a directed acyclic graph. So, repeat steps 1-2 until no vertex is left.

Example



$Q = \{ 0 \}$

All vertices of indegree 0 are placed on an initially empty queue.

OUTPUT: 0

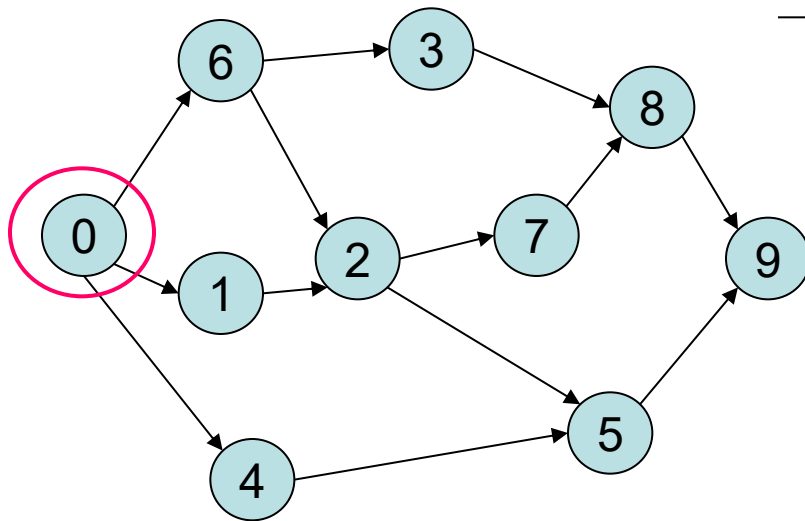
Adjacent list

0	→	6	1	4
1	→	2		
2	→	7	5	
3	→	8		
4	→	5		
5	→	9		
6	→	3	2	
7	→	8		
8	→	9		
9				

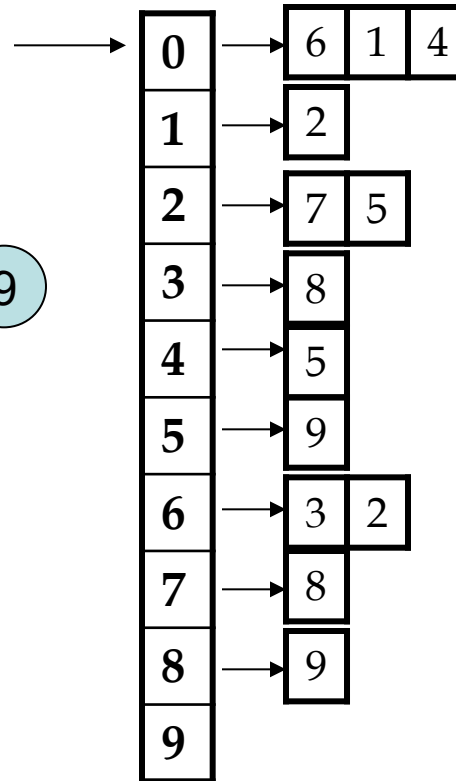
Indegree array

0	0
1	1
2	2
3	1
4	1
5	2
6	1
7	1
8	2
9	2

← start



Adjacent list



Indegree array

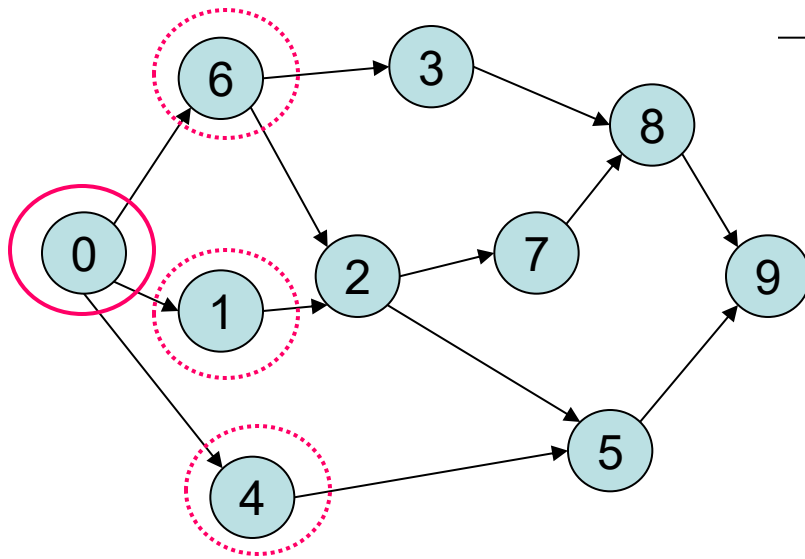
0	0	
1	1	-1
2	2	
3	1	
4	1	-1
5	2	
6	1	-1
7	1	
8	2	
9	2	

Dequeue 0 $Q = \{ \}$

→ remove 0's arcs – adjust
indegrees of neighbors (6, 1, 4)

OUTPUT: 0

Decrement 0's
neighbors, which are 6,
1 and 4.

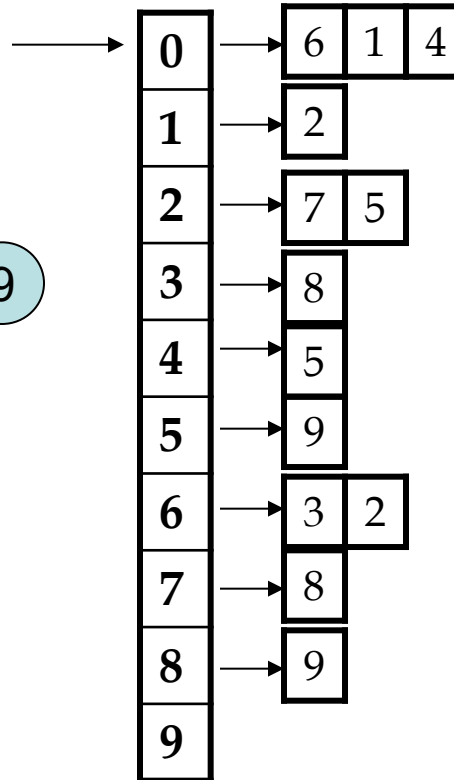


$Q = \{ 6, 1, 4 \}$

Enqueue all starting points

OUTPUT: 0

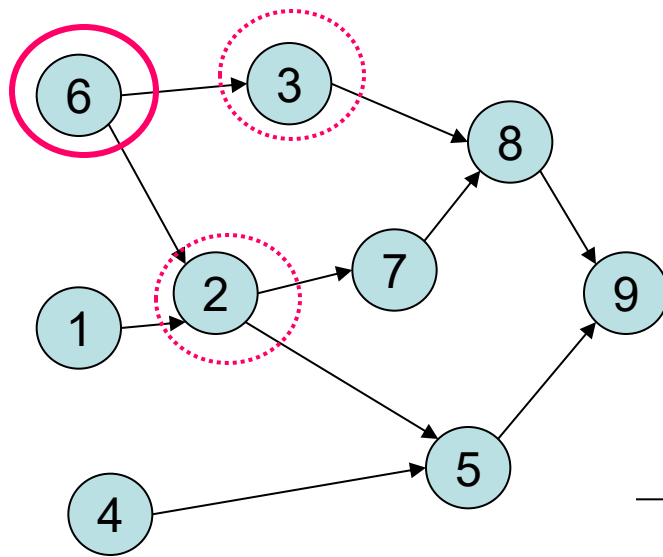
Adjacent list



Indegree array

0	0	
1	0	←
2	2	
3	1	
4	0	←
5	2	
6	0	←
7	1	
8	2	
9	2	

Enqueue all
new starting points
(indegree is 0)



Adjacent list

0	→	6	1	4
1	→	2		
2	→	7	5	
3	→	8		
4	→	5		
5	→	9		
6	→	3	2	
7	→	8		
8	→	9		
9				

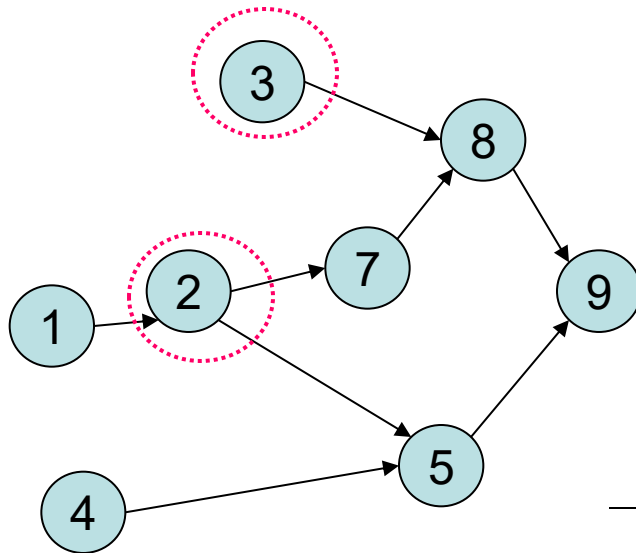
Indegree array

0	0	
1	0	
2	2	-1
3	1	-1
4	0	
5	2	
6	0	
7	1	
8	2	
9	2	

Dequeue 6 $Q = \{ 1, 4 \}$
 Remove arcs .. Adjust indegrees
 of neighbors, 3 and 2

OUTPUT: 0 6

Adjust indegrees of
 neighbors, which are
 3 and 2



Q = { 1, 4, 3 }
Enqueue 3

OUTPUT: 0 6

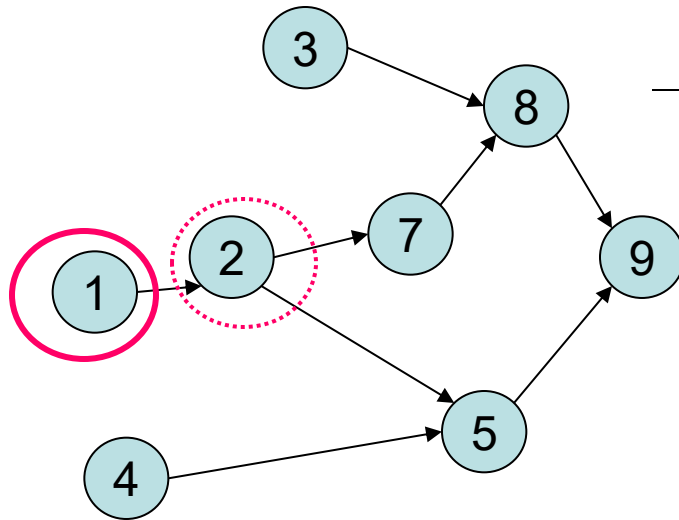
Adjacent list

0	→	6	1	4
1	→	2		
2	→	7	5	
3	→	8		
4	→	5		
5	→	9		
6	→	3	2	
7	→	8		
8	→	9		
9				

Indegree array

0	0
1	0
2	1
3	0
4	0
5	2
6	0
7	1
8	2
9	2

Enqueue new
starting point, which
is 3 alone.



Adjacent list

0	→	6	1	4
1	→	2		
2	→	7	5	
3	→	8		
4	→	5		
5	→	9		
6	→	3	2	
7	→	8		
8	→	9		
9				

Indegree array

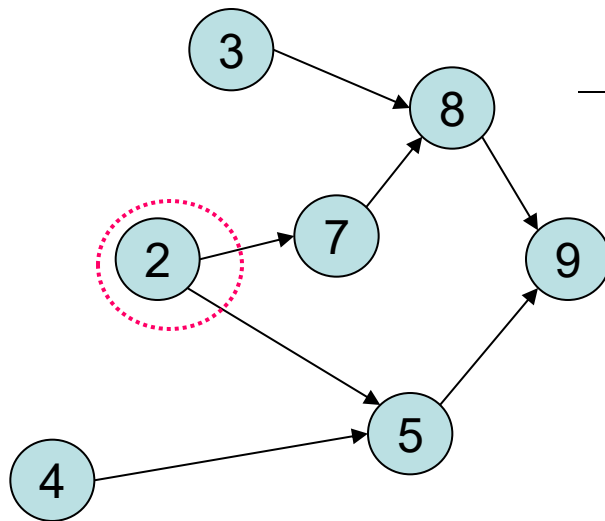
0	0
1	0
2	1
3	0
4	0
5	2
6	0
7	1
8	2
9	2

-1

Dequeue 1 $Q = \{ 4, 3 \}$
Adjust indegree of neighbor

OUTPUT: 0 6 1

Adjust indegree of neighbor, which is 2 alone.



Adjacent list

0	→	6	1	4
1	→	2		
2	→	7	5	
3	→	8		
4	→	5		
5	→	9		
6	→	3	2	
7	→	8		
8	→	9		
9				

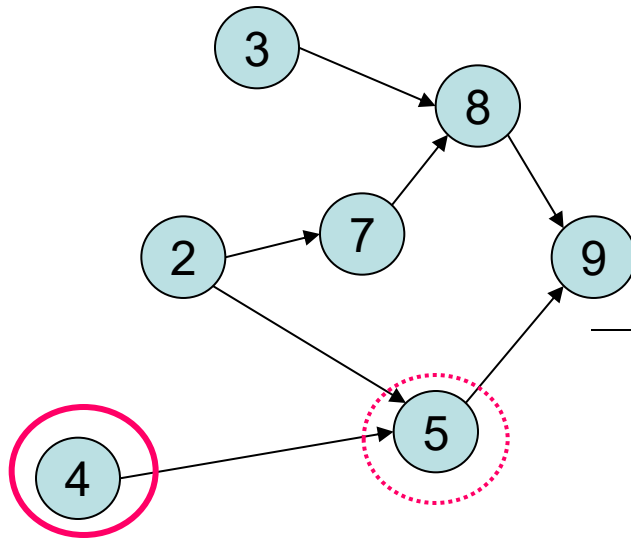
Indegree array

0	0
1	0
2	0
3	0
4	0
5	2
6	0
7	1
8	2
9	2

Dequeue 1 $Q = \{ 4, 3, 2 \}$
 Enqueue 2

OUTPUT: 0 6 1

Enqueue new
 starting point, which is
 2 alone.



Adjacent list

0	→	6	1	4
1	→	2		
2	→	7	5	
3	→	8		
4	→	5		
5	→	9		
6	→	3	2	
7	→	8		
8	→	9		
9				

Indegree array

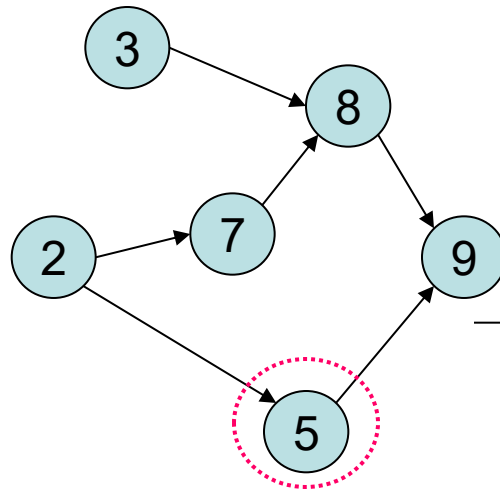
0	0
1	0
2	0
3	0
4	0
5	2
6	0
7	1
8	2
9	2

-1

Dequeue 4 $Q = \{ 3, 2 \}$
Adjust indegree of neighbor

Adjust 4's neighbor,
which is 5

OUTPUT: 0 6 1 4



Adjacent list

0	→	6	1	4
1	→	2		
2	→	7	5	
3	→	8		
4	→	5		
5	→	9		
6	→	3	2	
7	→	8		
8	→	9		
9				

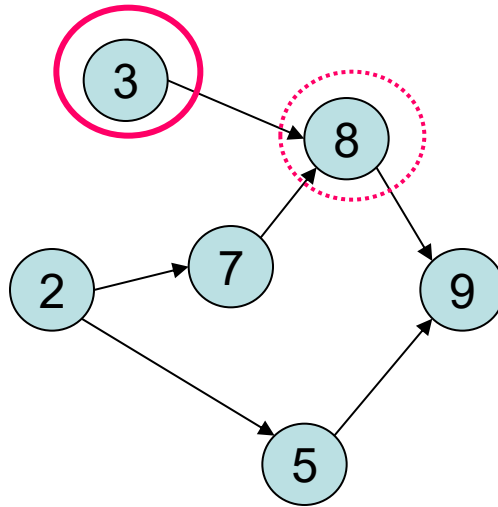
Indegree array

0	0
1	0
2	0
3	0
4	0
5	1
6	0
7	1
8	2
9	2

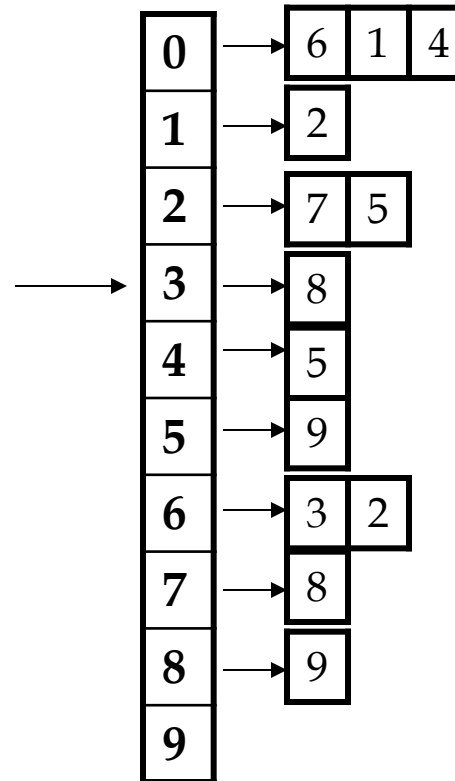
Dequeue 4 $Q = \{ 3, 2 \}$
 No new starting point found

NO new starting
 point

OUTPUT: 0 6 1 4



Adjacent list



Indegree array

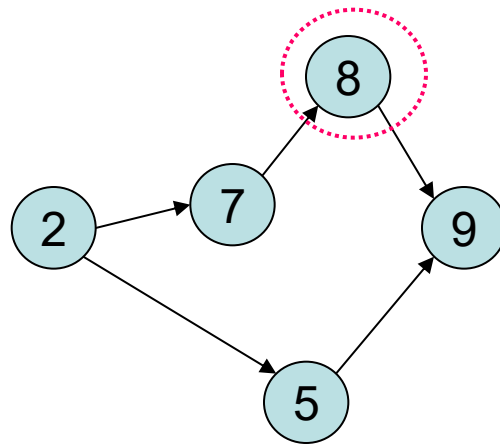
0	0
1	0
2	0
3	0
4	0
5	1
6	0
7	1
8	2
9	2

-1

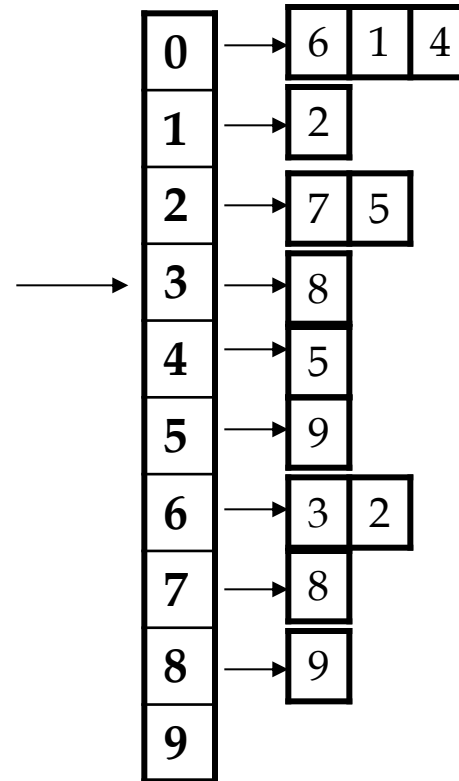
Dequeue 3 $Q = \{ 2 \}$

Adjust 3's neighbor, which is 8 alone.

OUTPUT: 0 6 1 4 3



Adjacent list

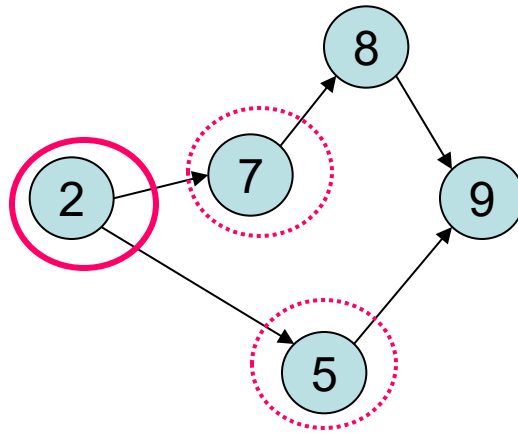


Indegree array

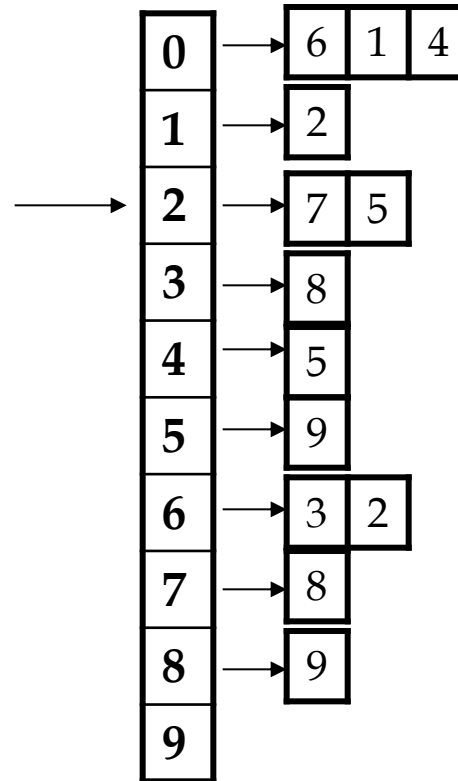
0	0
1	0
2	0
3	0
4	0
5	1
6	0
7	1
8	1
9	2

Dequeue 3 $Q = \{ 2 \}$
 No new starting point found

OUTPUT: 0 6 1 4 3



Adjacent list



Indegree array

0	0
1	0
2	0
3	0
4	0
5	1
6	0
7	1
8	1
9	2

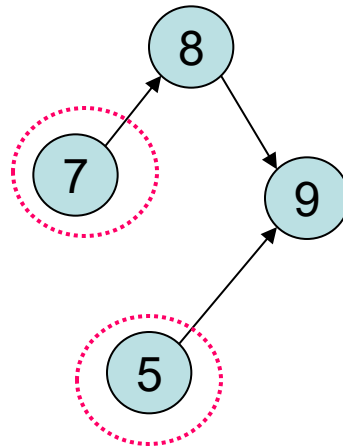
-1

-1

Dequeue 2 $Q = \{ \}$

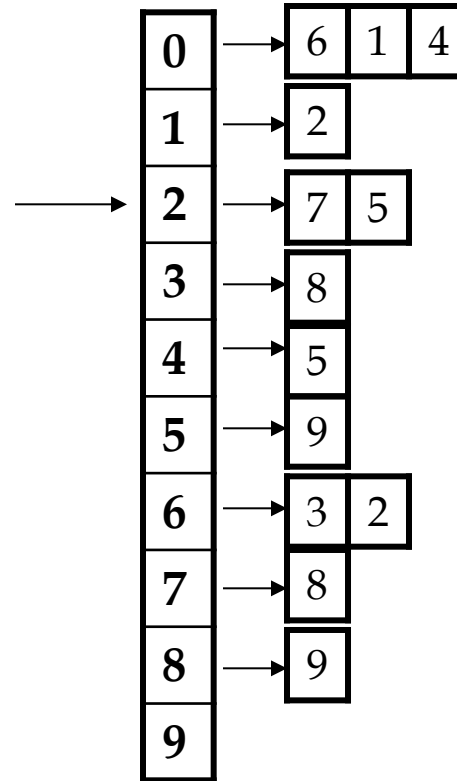
Adjust 2's neighbors, which are 7 and 5.

OUTPUT: 0 6 1 4 3 2



Dequeue 2 $Q = \{7, 5\}$
 Enqueue 7, 5

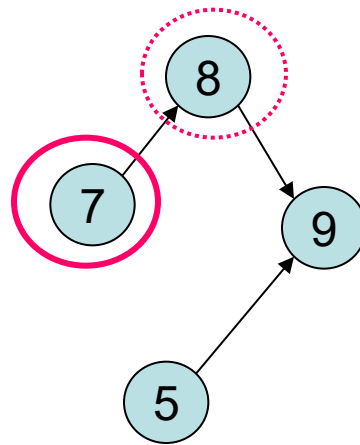
Adjacent list



Indegree array

0	0	
1	0	
2	0	
3	0	
4	0	
5	0	←
6	0	
7	0	←
8	1	
9	2	

OUTPUT: 0 6 1 4 3 2



Adjacent list

0	→	6	1	4
1	→	2		
2	→	7	5	
3	→	8		
4	→	5		
5	→	9		
6	→	3	2	
7	→	8		
8	→	9		
9				

Indegree array

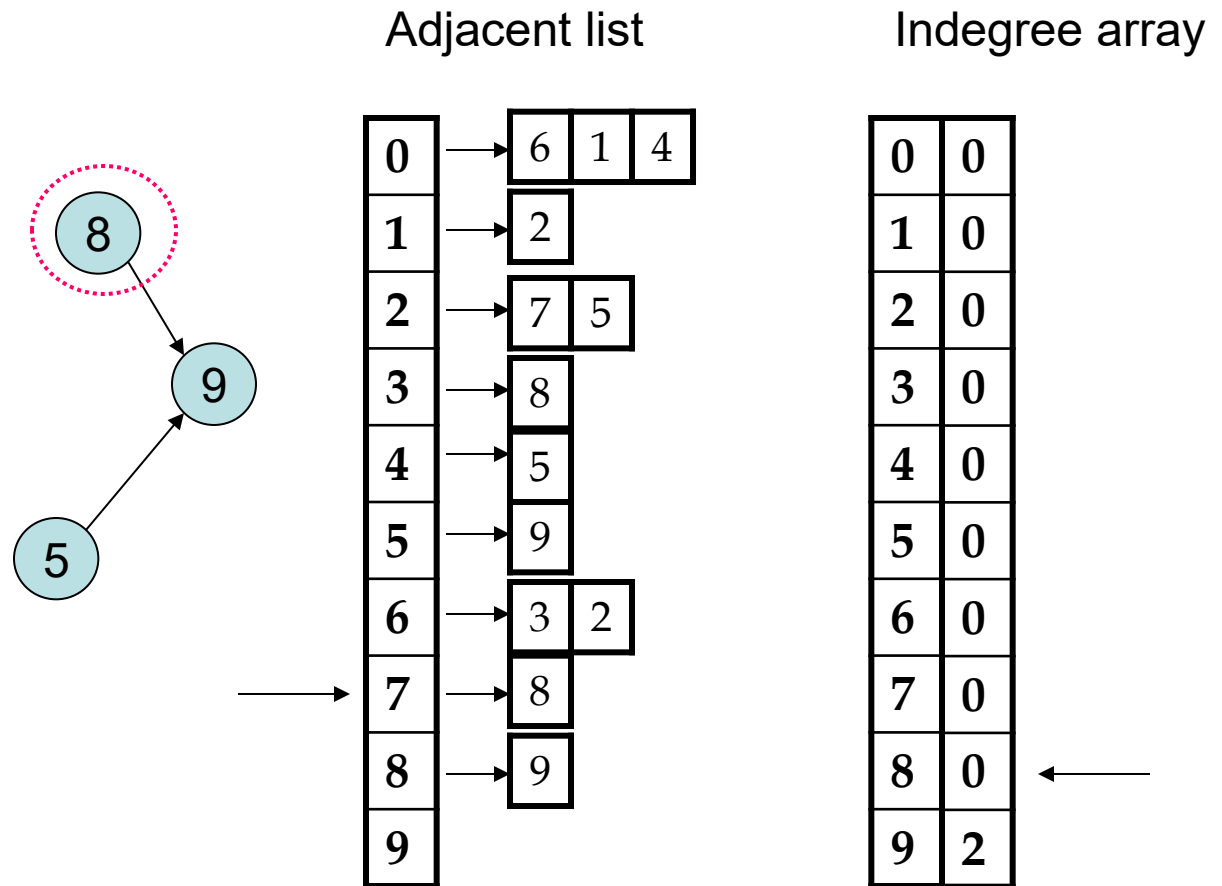
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	1
9	2

-1

Dequeue 7 $Q = \{ 5 \}$

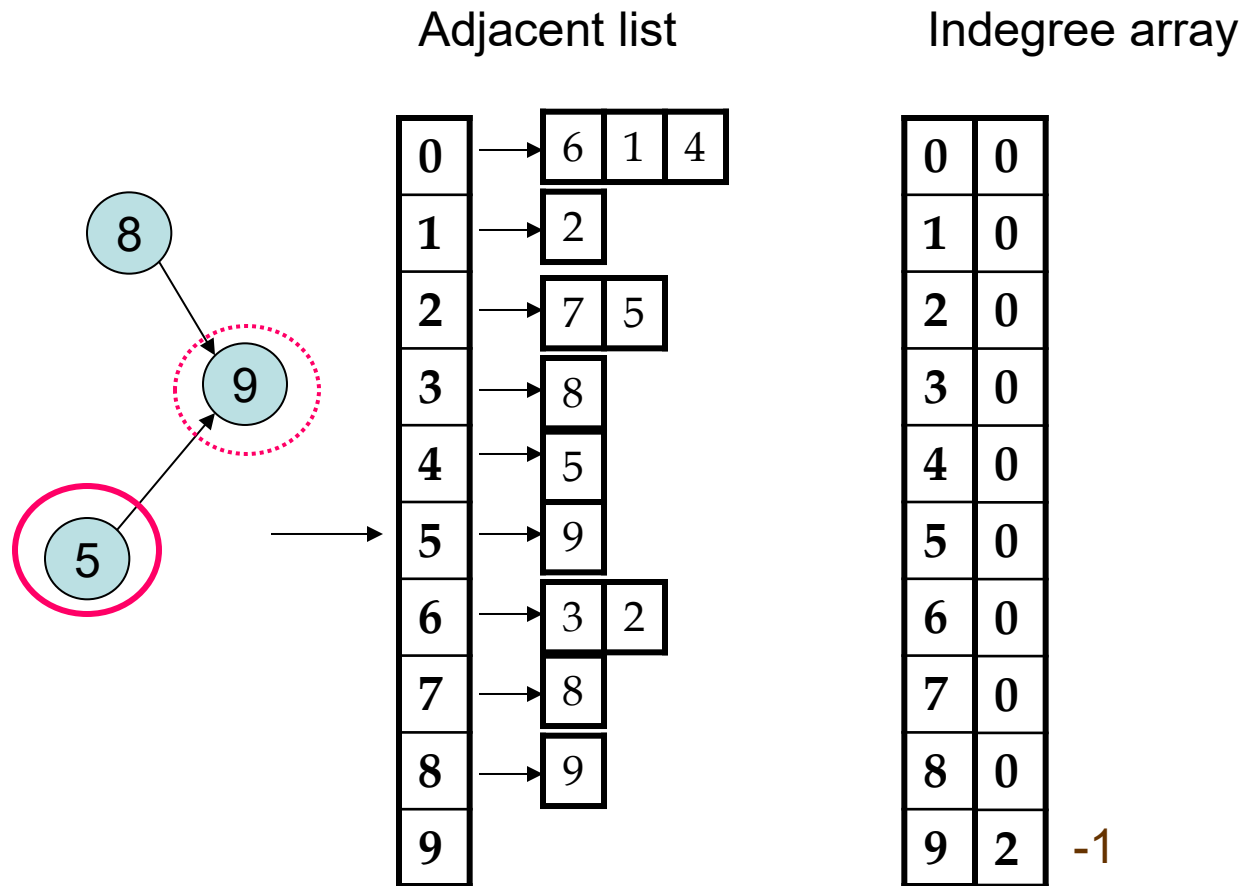
Adjust indegree of neighbor, which is 8.

OUTPUT: 0 6 1 4 3 2 7



Dequeue 7 $Q = \{ 5, 8 \}$
 Adjust indegree of neighbor, which is 8.

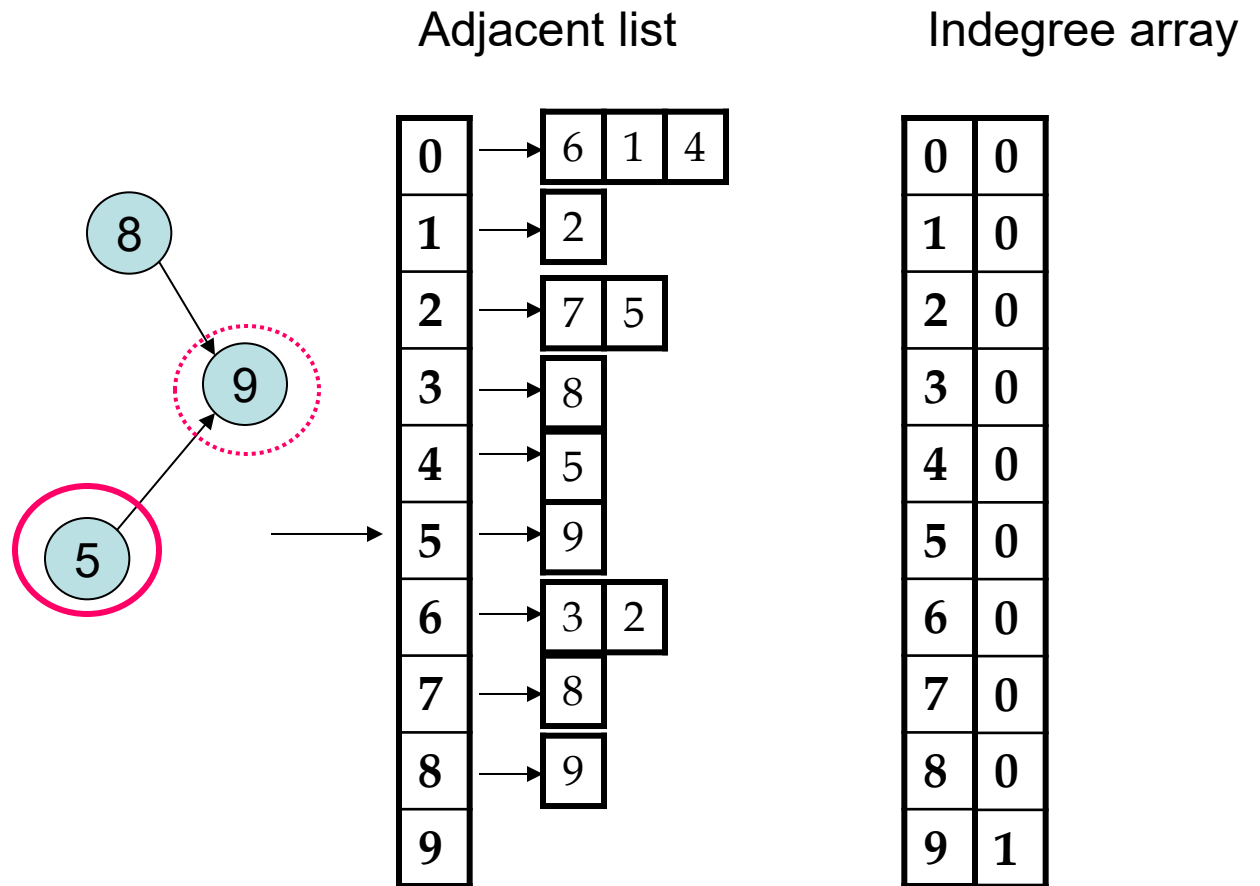
OUTPUT: 0 6 1 4 3 2 7



Dequeue 5 $Q = \{ 8 \}$

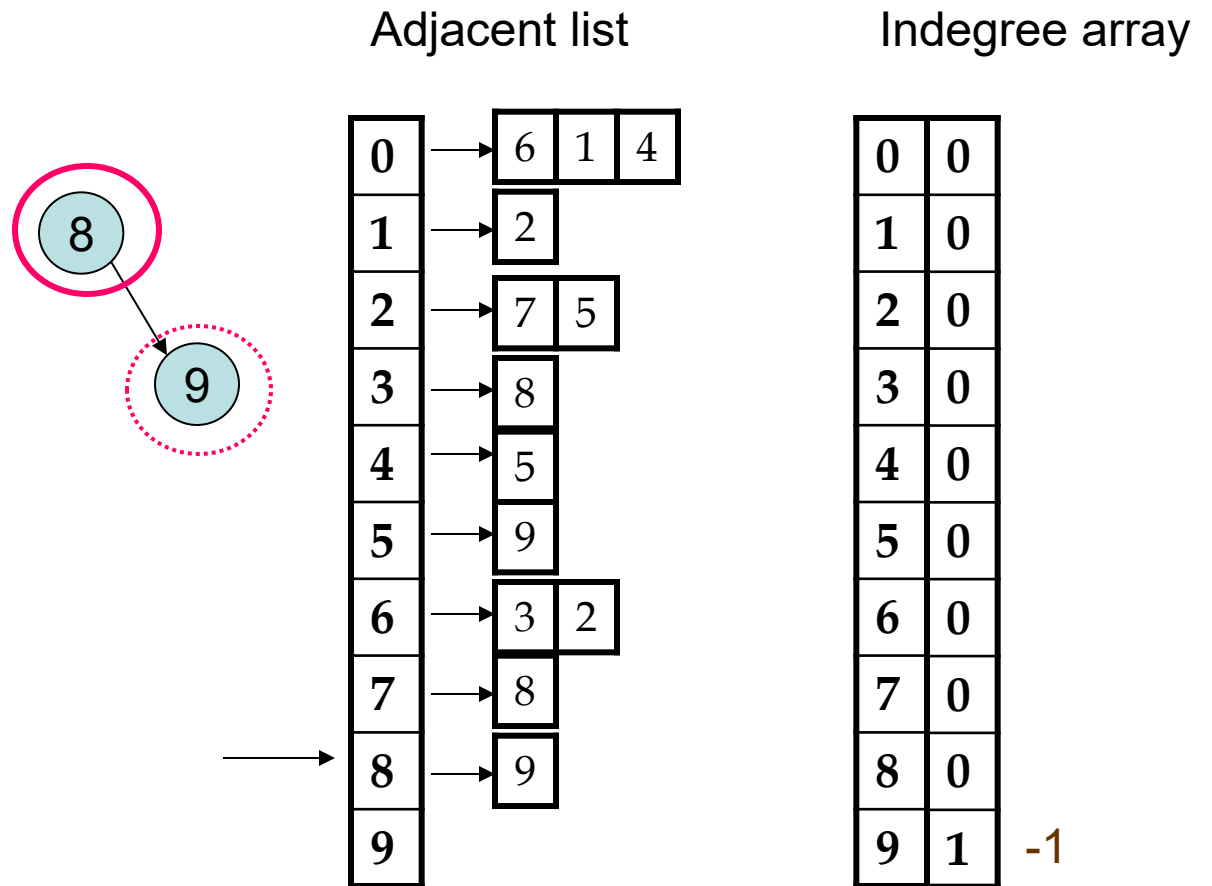
Adjust indegree of neighbor, which is 9.

OUTPUT: 0 6 1 4 3 2 7 5



Dequeue 5 $Q = \{ 8 \}$
No new starting point found

OUTPUT: 0 6 1 4 3 2 7 5



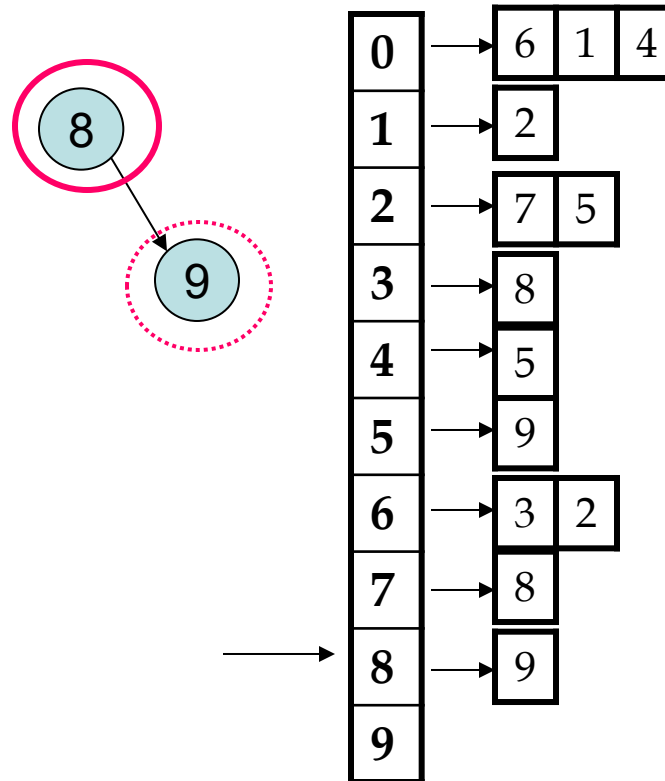
Dequeue 8 $Q = \{ \}$

Adjust indegree of neighbor, which is 9.

OUTPUT: 0 6 1 4 3 2 7 5 8

Adjacent list

Indegree array



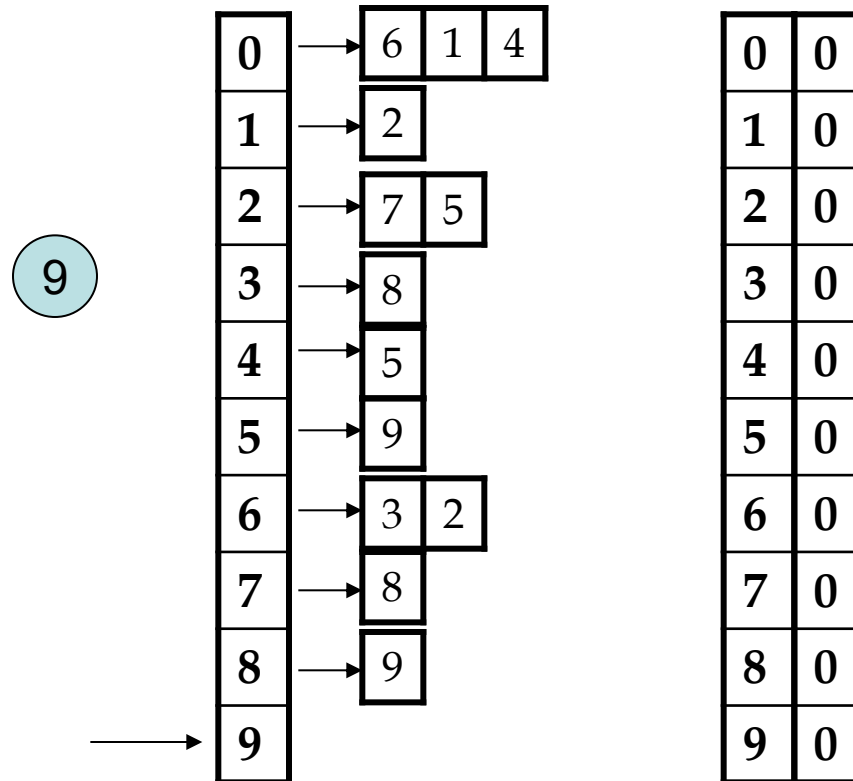
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

Dequeue 8 $Q = \{9\}$
 Enqueue 9.

OUTPUT: 0 6 1 4 3 2 7 5 8

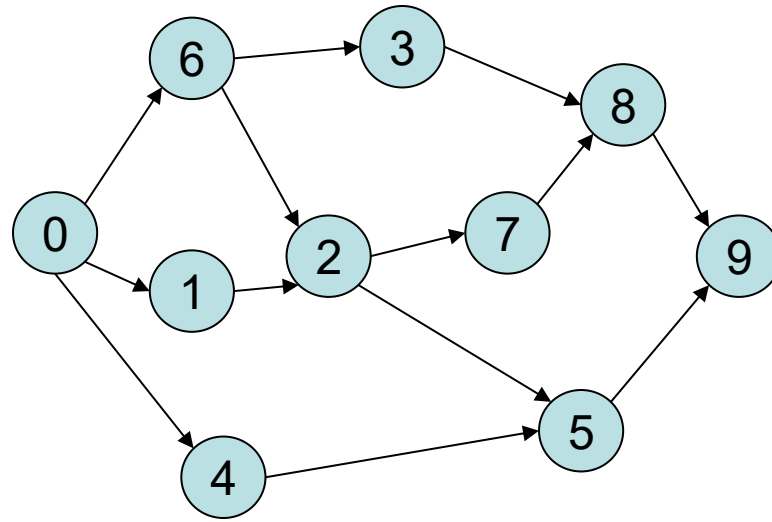
Adjacent list

Indegree array



Dequeue 9 $Q = \{ \}$
 STOP – no neighbors

OUTPUT: 0 6 1 4 3 2 7 5 8 9



OUTPUT: 0 6 1 4 3 2 7 5 8 9

Is output topologically correct?

Topological Sort: Complexity

- We never visited a vertex more than one time.
- For each vertex,
 - ♦ we had to examine all outgoing edges,
 - ♦ it took time proportional to $\text{outdegree}(v) + 1$.
- Since it is summed over all vertices, the running time is $O(n + m)$ if there are n vertices and m arcs.